

Robustness of Image-Processing and Vision Methods to Image Corruption

How much do corrupted images hurt four vision tasks — and how much of the damage can classical restoration and model fine-tuning undo?

4

vision tasks

low-level + deep models

3

corruption types

noise × 2, motion blur

3

severity levels

SNR logged per image

2

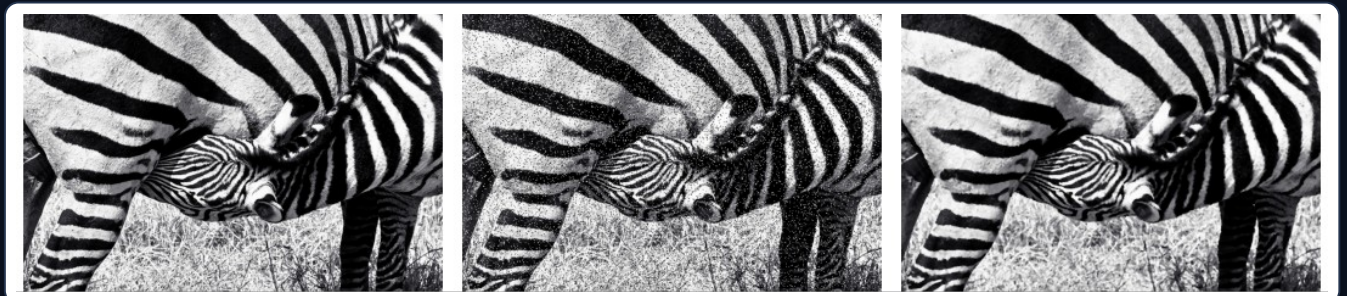
recovery strategies

enhance · fine-tune

Ohad Shpizhizen · Shira Tziony

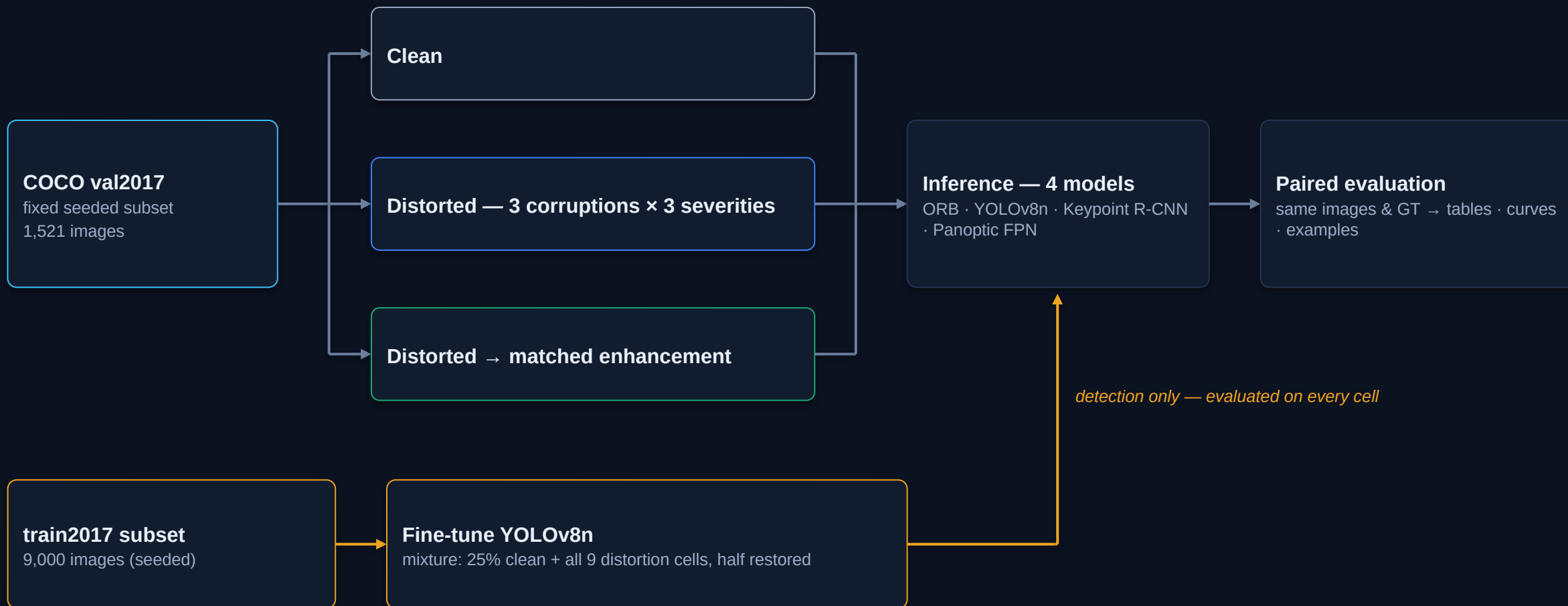
COCO val2017 — fixed 1,521-image evaluation subset

Paired comparisons: identical images & GT in every cell



clean → corrupted (salt & pepper, high) → restored (median 3×3)

One fixed image set — every variant measured against it



SNR + degradation parameters logged per image · distorted & enhanced images stored as lossless PNG · every cell reproducible via seeded RNG

COCO val2017 — one seeded subset behind every number

1,521

evaluation images — seeded sample of 1,500, topped up with 21 for class coverage

11,129

ground-truth instances in the subset · 3,273 person instances

$r = 0.999$

class-distribution correlation with the full 5,000-image val2017

9,000

train2017 images (seeded) — used only for fine-tuning

WHY THIS SETUP

- Public benchmark: 80 classes, ground truth for boxes, person keypoints and panoptic masks
- The same images serve all four tasks — no cross-dataset alignment
- Paired comparisons: identical images & GT for every variant, so differences come only from image quality
- Custom subset $\Rightarrow$ published full-val2017 scores are approximate sanity checks, not direct comparisons

Class coverage: every category reaches ≥ 20 GT instances, except toaster (9) and hair drier (11) — both exhaust their entire val2017 supply.

Four tasks — from raw pixel structure to full scene understanding

TASK	LEVEL	MODEL / ALGORITHM	METRIC	WHAT IT MEASURES
Feature matching	Low-level	ORB + BF matcher (Hamming, cross-check) — OpenCV	Match ratio vs clean	Share of clean keypoints re-matched in the degraded image (Hamming ≤ 64) — does local structure survive?
Object detection	High-level · DL	YOLOv8n, COCO-pretrained — ultralytics	mAP@[.5:.95]	Detect + tightly localize 80 classes; AP averaged over 10 IoU thresholds
Keypoint detection	High-level · DL	Keypoint R-CNN, ResNet50-FPN — torchvision	OKS AP	17 human joints per person; AP with IoU replaced by scale-normalized keypoint similarity
Panoptic segmentation	High-level · DL	Panoptic FPN R50 — detectron2	PQ = SQ × RQ	Every pixel gets category + segment: mask quality (SQ) × segment recognition (RQ)

All metrics $\in [0, 1]$, higher is better. ORB needs no ground truth — each clean image is its own reference. The three learned models are COCO-pretrained; detection is the focus of the fine-tuning comparison.

Three corruptions, three severities — one matched repair each

CORRUPTION	SEVERITY → MEAN SNR	MATCHED ENHANCEMENT
Gaussian noise — sensor noise; variance sampled & recorded per image	19.5 / 13.9 / 10.0 dB	Non-blind BM3D — σ derived from the recorded variance
Salt & pepper — impulse pixels → 0 or 255; amount 1% / 5% / 12%	18.7 / 11.8 / 8.2 dB	Median filter 3×3 — discards isolated outliers
Motion blur — global linear kernel 5 / 11 / 21 px; angle sampled & recorded	20.8 / 17.7 / 15.6 dB	Non-blind Wiener deconvolution — kernel rebuilt from recorded size + angle

Severities: low / medium / high. Recorded degradation parameters enable matched, non-blind restoration — no estimation from the corrupted image. All variants stored as lossless PNG.

Clean

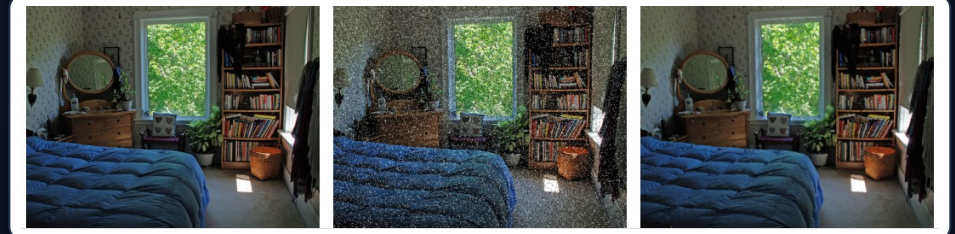
Distorted (high)

Enhanced

Gaussian noise → BM3D



Salt & pepper → median 3×3



Motion blur → Wiener



Pretrained models on clean images — the reference for every cell

Feature matching

ORB match ratio

1.000

by construction — each clean image is matched to itself

Object detection

YOLOv8n · mAP@[.5:.95]

0.352

$\Delta -0.021$

published full val2017 ≈ 0.373

Keypoint detection

Keypoint R-CNN · OKS AP

0.657

$\Delta +0.007$

published full val2017 ≈ 0.650

Panoptic segmentation

Panoptic FPN · PQ

0.410

$\Delta -0.005$

published ≈ 0.415 · SQ 0.775 · RQ 0.500

Sanity check, not benchmark replication. Published scores use the full 5,000-image val2017; ours use the 1,521-image subset. Differences of -0.021 to $+0.007$ indicate no issue in annotation handling, format conversion, or evaluation.

Enhancement recovers ground in every one of the 36 cells

■ distorted ■ enhanced ■ fine-tuned (detection only)

dashed line = clean baseline · bar labels = share of damage recovered

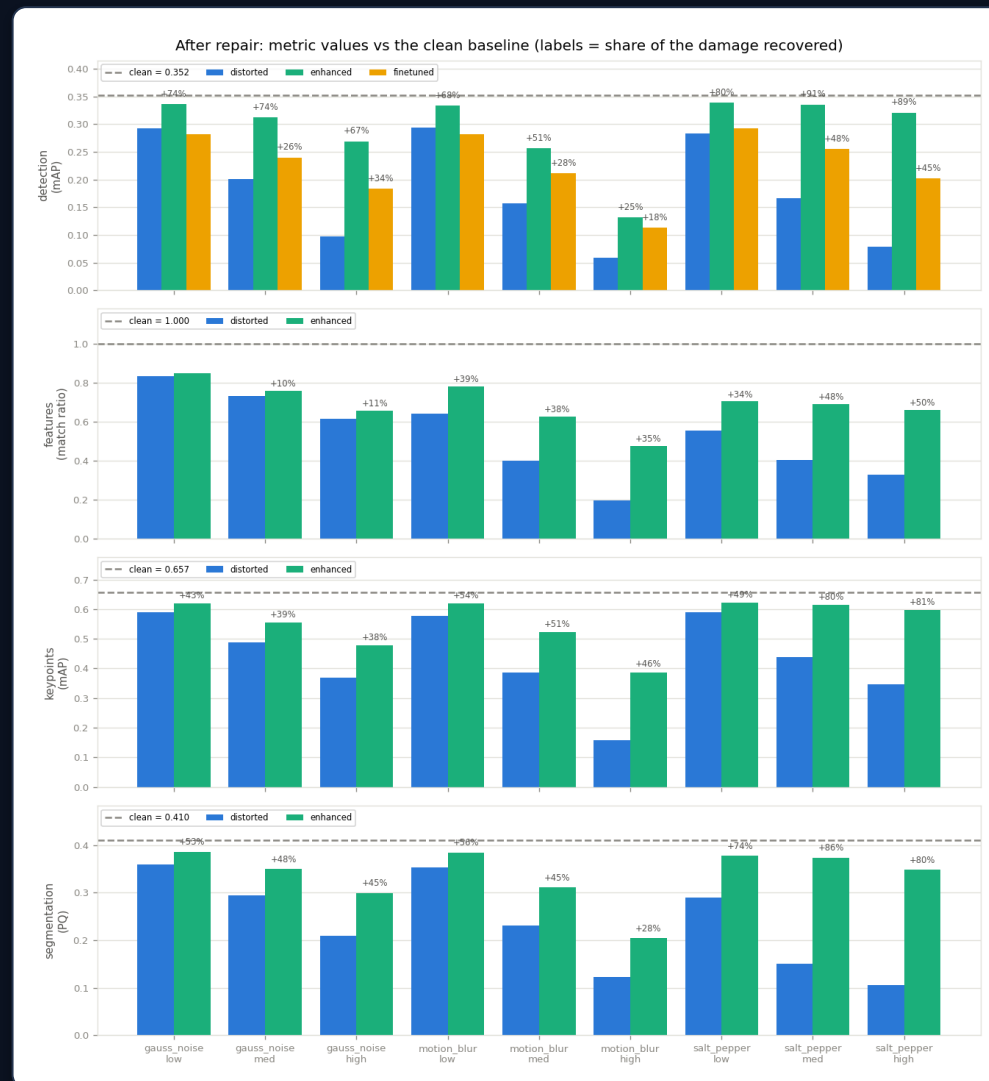
- Corruption hurts every task, and more severity hurts more — detection falls as far as -0.293 mAP (motion blur, high).
- Classical enhancement improves the raw distorted score in all 36 task × corruption × severity cells.
- For detection, enhancement is the best standalone repair in all 9 cells — ahead of fine-tuning alone.
- Recovery is largest for salt & pepper (up to +91% of the damage), smallest for heavy motion blur.

36 / 36

cells improved by classical enhancement

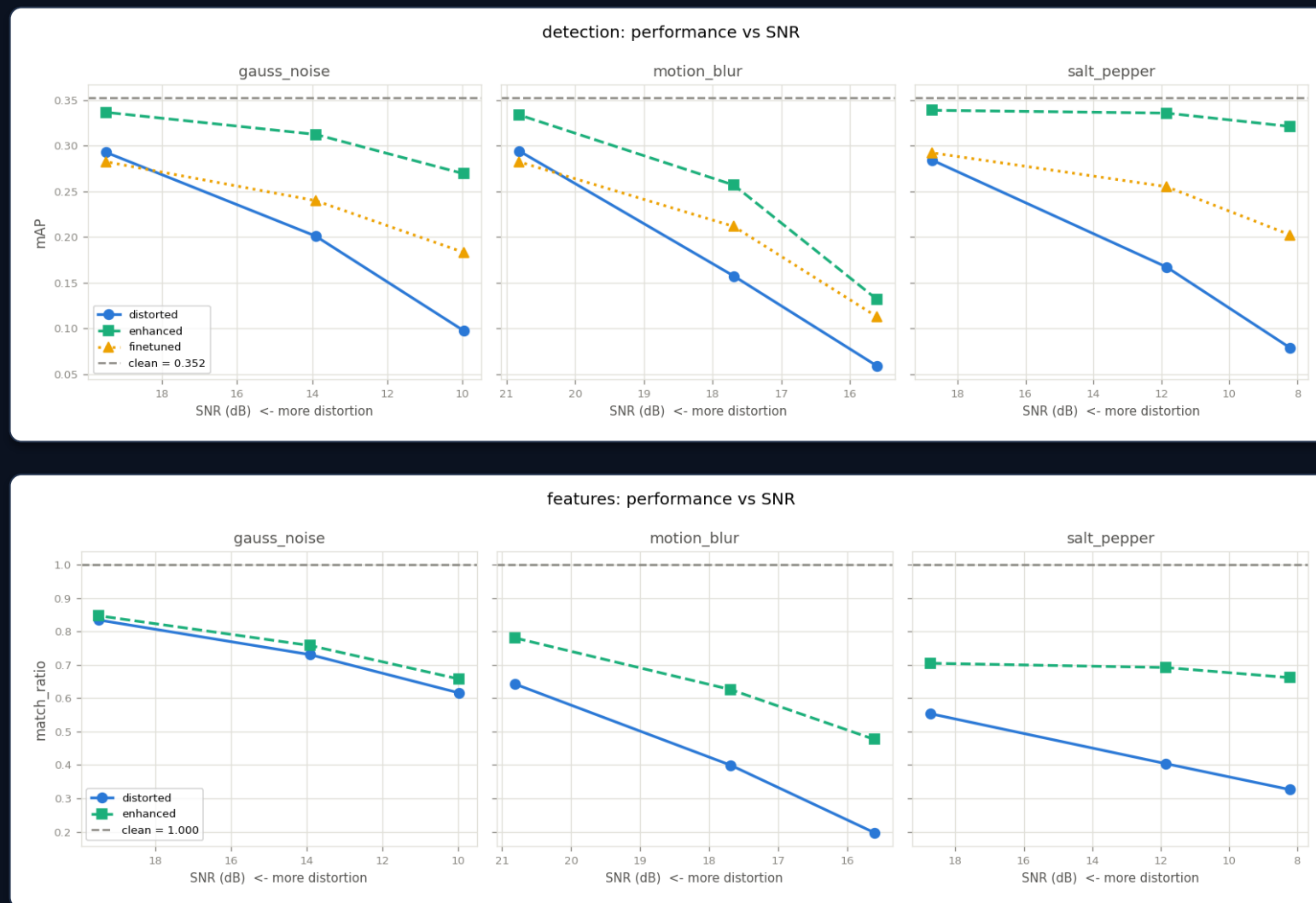
9 / 9

detection cells where enhancement is the best standalone repair



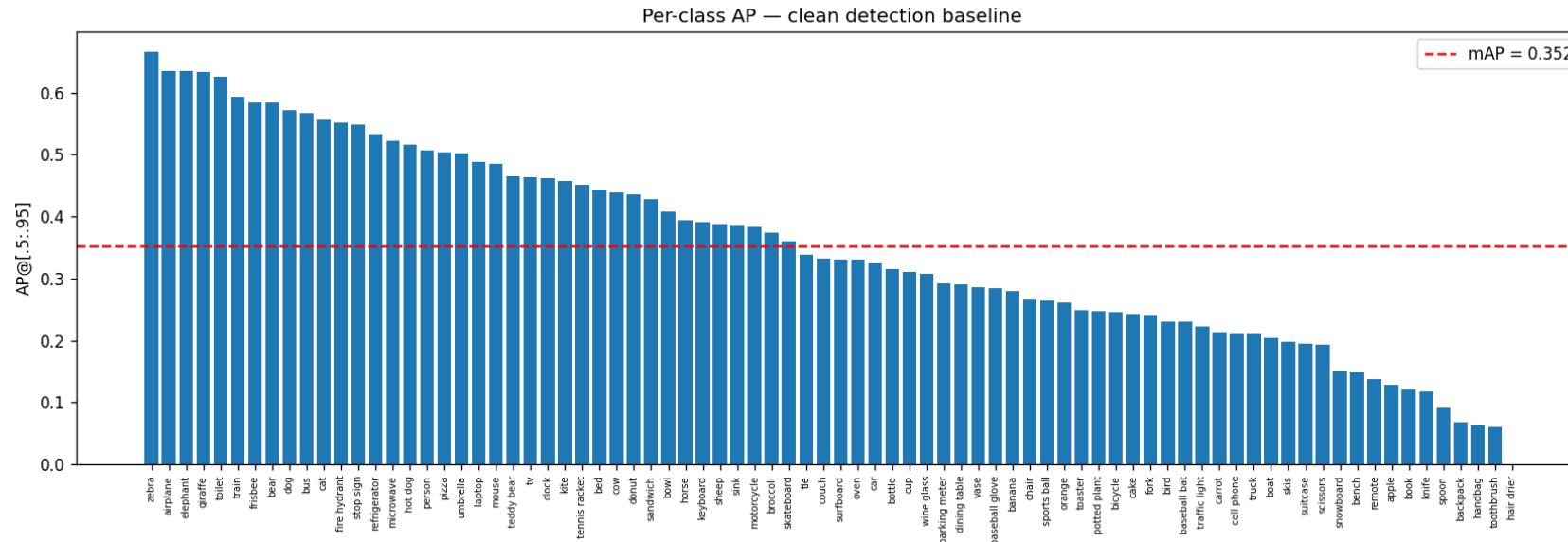
SNR tracks severity within a corruption — not damage across corruptions

- Within each corruption type, every task degrades monotonically as mean SNR falls.
- Across types SNR misleads: high motion blur (15.6 dB) costs ORB -0.80 and keypoints -0.50 — worse than high noise at 10.0 / 8.2 dB.
- Interpretation: geometry-sensitive tasks depend on spatial structure, which blur destroys — pixel-wise SNR does not capture that.

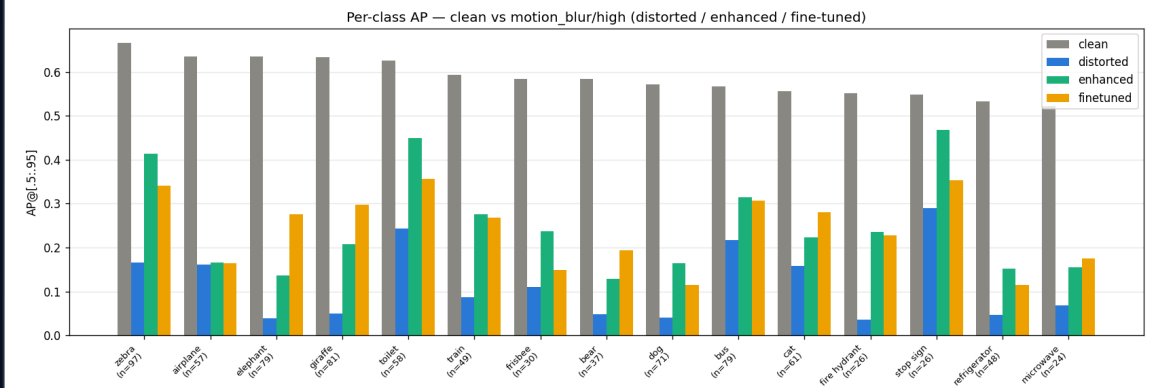
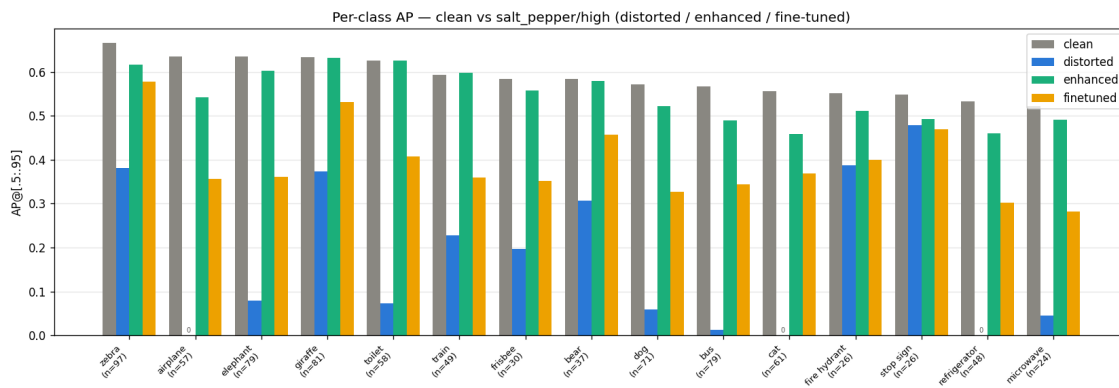


Keypoint & segmentation curves → Appendix B

Which classes break — and which repair brings them back



- Top: clean AP for all 80 classes; dashed line = mAP 0.352.
- Bottom: the 15 classes with the highest clean AP, at high severity; n = GT instances.
- gray clean · blue distorted · green enhanced · amber fine-tuned
- Gaussian-noise comparison → Appendix B.



What the numbers look like on real images

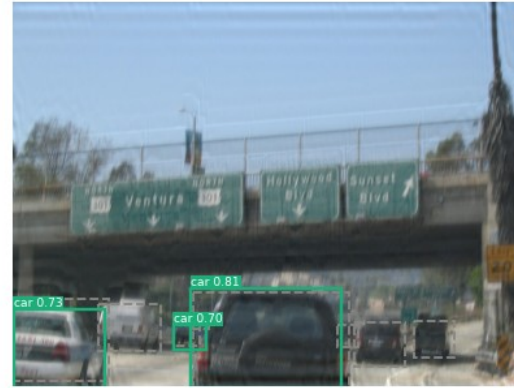
Clean + YOLO



Distorted — motion blur (high)



Enhanced — Wiener



Fine-tuned YOLO on distorted



Clean + Keypoint R-CNN



Distorted — Gaussian noise (high)



Enhanced — BM3D



READING THE PANELS

- Gray dashed = ground truth; solid boxes / white skeletons + yellow joints = predictions.
- Blur wipes out cars and signs; deblurring brings detections back — fine-tuning recovers some directly.
- Noise drags joints off body parts; after BM3D the skeletons re-attach.

Corruption breaks structure — matched repair rebuilds most of it

Ground truth

Clean · PQ 0.410

Distorted · PQ 0.105

Enhanced · PQ 0.348

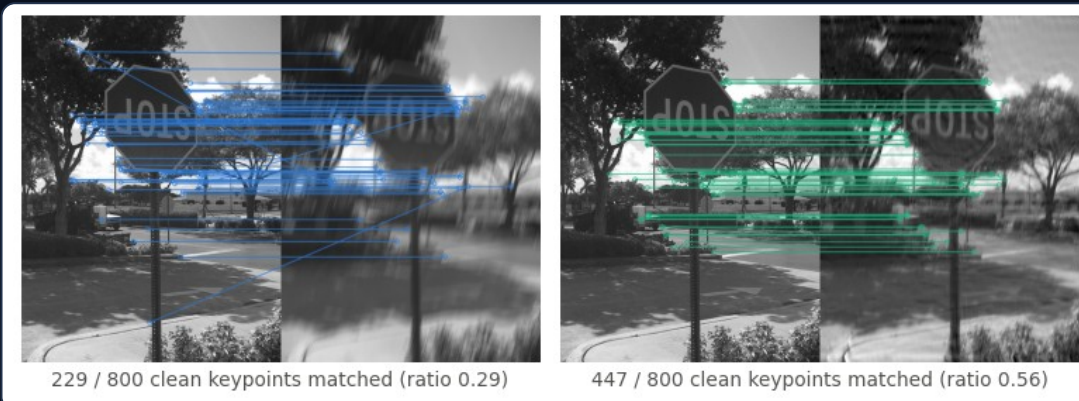


PANOPTIC — SALT & PEPPER, HIGH

- Color = semantic category, white hairlines = segment boundaries; PQ above each panel.
- Impulse noise shatters the scene into false segments: PQ 0.410 → 0.105.
- The 3×3 median filter rebuilds the layout — PQ back to 0.348.

Clean ↔ Distorted (blur high)

Clean ↔ Enhanced (Wiener)

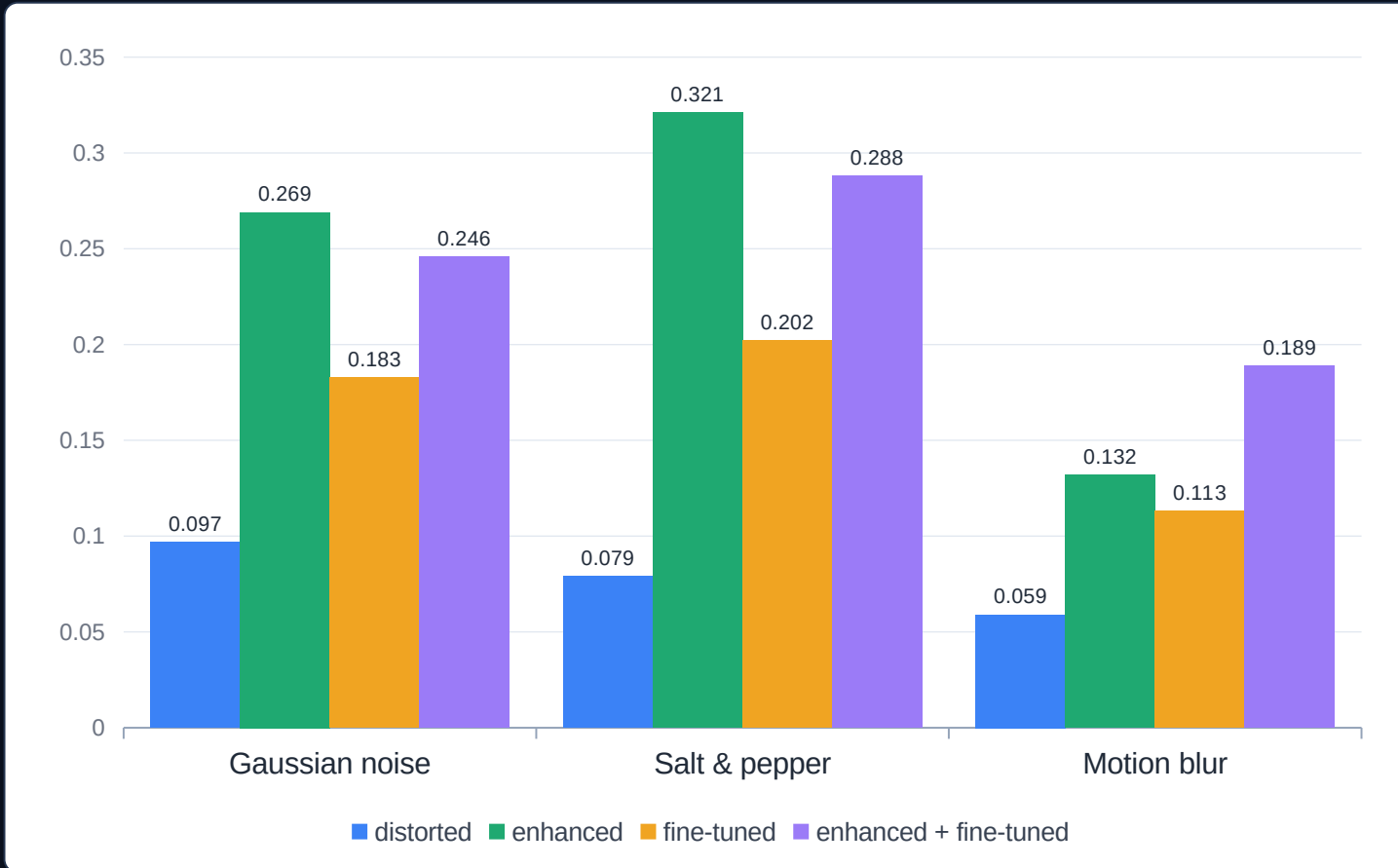


ORB MATCHING — MOTION BLUR, HIGH

- Lines join good matches (Hamming ≤ 64 , cross-checked — the metric's criterion); caption = per-image match ratio.
- Heavy blur leaves few good matches; Wiener deconvolution lifts these two examples from 0.29 / 0.05 to 0.56 / 0.39.

Enhancement wins alone — stacking wins on heavy motion blur

mAP@[.5:.95] at high severity — clean baseline = 0.352



Same ranking at low & medium severity for the standalone repairs — full table in Appendix A.

9 / 9

detection cells where enhancement beats fine-tuning alone

7 / 9

corrupted cells where fine-tuning beats the pretrained detector

44.5%

of the high-blur damage recovered by the stack — vs 24.9% enhancement-only, 18.3% fine-tuning-only

0.3521 → 0.3104

clean mAP after fine-tuning (−0.0417) — the price of robustness

Stacked repair achieves the best mAP for medium and high motion blur.

Fine-tuning pays off once the domain gap is large

Measured — mean fine-tuning recovery vs pretrained (mAP)

LOW SEVERITY

gauss -0.010 · blur -0.012 · S&P $+0.008$

-0.005

MEDIUM SEVERITY

gauss $+0.039$ · blur $+0.054$ · S&P $+0.088$

$+0.060$

HIGH SEVERITY

gauss $+0.086$ · blur $+0.054$ · S&P $+0.124$

$+0.088$

Monotonic for both noise types; motion blur plateaus at $\approx +0.054$ for medium and high.

WHY — INTERPRETATION

- Low severity: inputs sit close to the clean training domain — the pretrained detector is still strong, so adaptation has little headroom.
- Fine-tuning trades clean accuracy (-0.042 mAP) for corruption robustness; at low severity that trade can net out slightly negative.
- Medium / high severity: the domain gap grows, so training on corrupted + restored examples pays off ($+0.060$ / $+0.088$).
- The motion-blur plateau marks the limit of adaptation: no amount of fine-tuning reconstructs spatial information the blur kernel destroyed.

Match the repair to the corruption — and know when to stack

TAKEAWAYS — MEASURED

1. Matched classical enhancement is the strongest standalone detection repair in all 9 cells — and improves all 36 cells across the four tasks.
2. One enhancement pipeline generalized to all four tasks with no retraining; fine-tuning applied only to YOLOv8n detection.
3. Fine-tuning is complementary: most valuable at high severity, and best overall when stacked on medium / high motion blur.
4. SNR ranks severities within a corruption family, but not damage across mechanisms — structure loss is what hurts geometry-sensitive tasks.

LIMITATIONS

- Synthetic, global, spatially uniform corruptions only
- Restoration is non-blind — BM3D / Wiener use recorded corruption parameters
- Fine-tuning evaluated only for YOLOv8n detection
- Mixed, spatially varying, or unseen real-world corruptions were not evaluated — conclusions do not extend to them

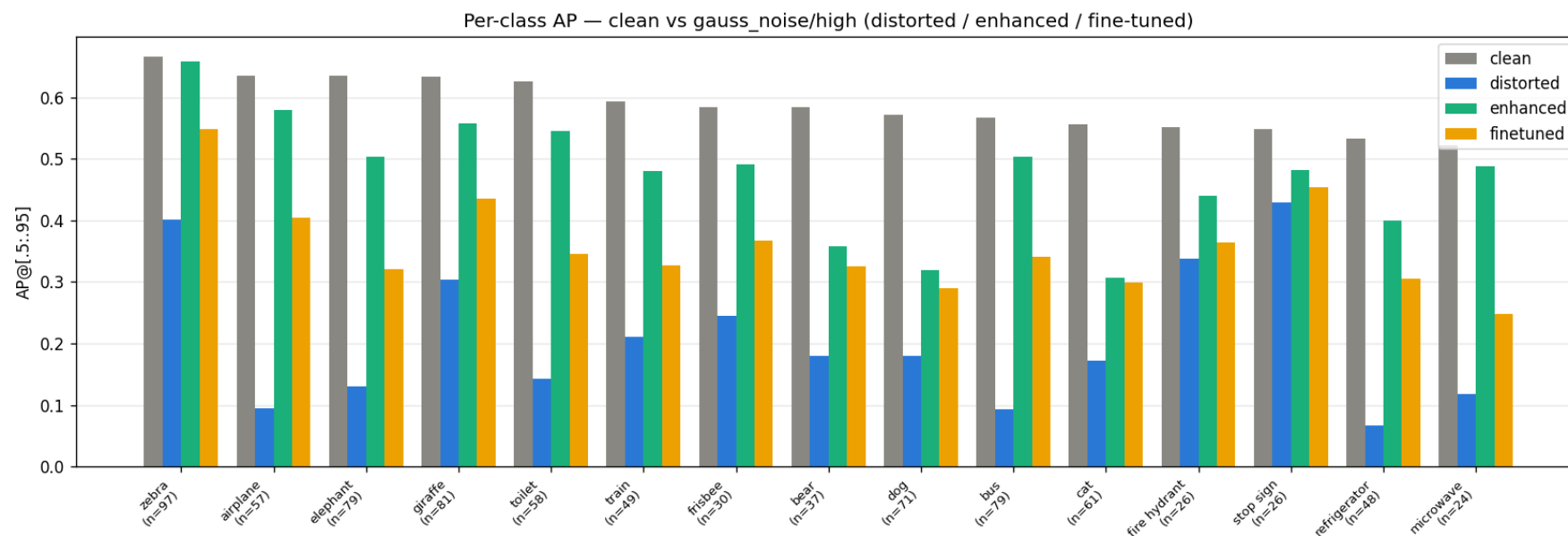
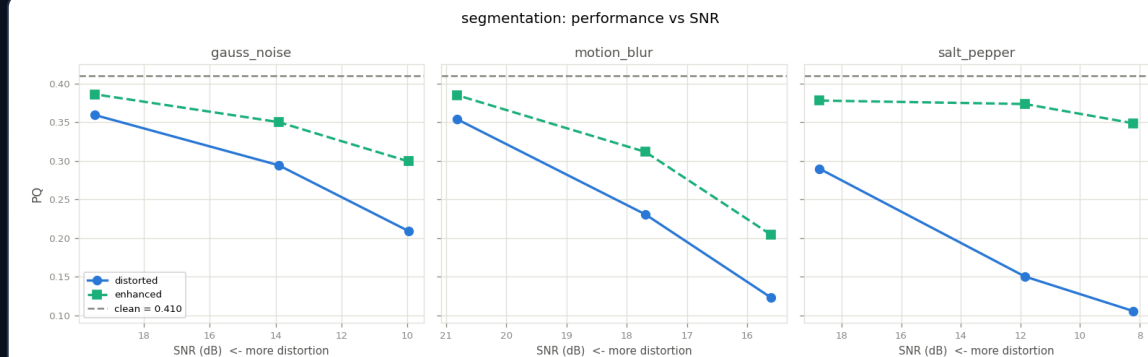
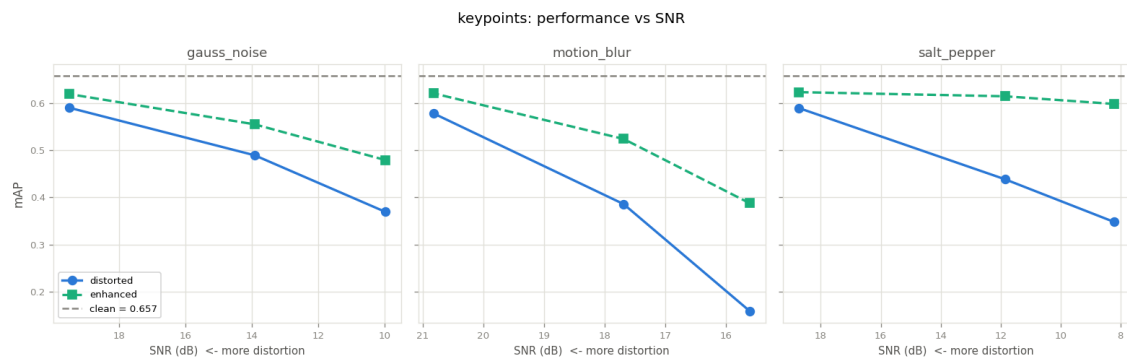
Practical rule of thumb: corruption known & parameterizable → restore classically · heavy, partly irreversible → stack both · unknown at inference → adapt the model.

Full results — every task × corruption × severity cell

task	distortion	sev	SNR (dB)	clean	distorted	enhanced	fine-tuned	ft + enh	degradation	rec (enh)	rec (FT)	rec (comb)
detection	gauss_noise	low	19.5	0.352	0.293	0.336	0.282	0.299	-0.059	+0.044	-0.010	+0.006
detection	gauss_noise	med	13.9	0.352	0.201	0.312	0.240	0.279	-0.151	+0.111	+0.039	+0.078
detection	gauss_noise	high	10.0	0.352	0.097	0.269	0.183	0.246	-0.255	+0.172	+0.086	+0.149
detection	motion_blur	low	20.8	0.352	0.294	0.334	0.282	0.299	-0.058	+0.039	-0.012	+0.005
detection	motion_blur	med	17.7	0.352	0.157	0.257	0.211	0.261	-0.195	+0.100	+0.054	+0.104
detection	motion_blur	high	15.6	0.352	0.059	0.132	0.113	0.189	-0.293	+0.073	+0.054	+0.131
detection	salt_pepper	low	18.7	0.352	0.284	0.339	0.292	0.299	-0.068	+0.055	+0.008	+0.015
detection	salt_pepper	med	11.8	0.352	0.167	0.336	0.255	0.297	-0.185	+0.169	+0.088	+0.130
detection	salt_pepper	high	8.2	0.352	0.079	0.321	0.202	0.288	-0.274	+0.242	+0.124	+0.210
features	gauss_noise	low	19.5	1.000	0.834	0.847	—	—	-0.166	+0.013	—	—
features	gauss_noise	med	13.9	1.000	0.731	0.758	—	—	-0.269	+0.027	—	—
features	gauss_noise	high	10.0	1.000	0.616	0.658	—	—	-0.384	+0.042	—	—
features	motion_blur	low	20.8	1.000	0.643	0.781	—	—	-0.357	+0.138	—	—
features	motion_blur	med	17.7	1.000	0.399	0.626	—	—	-0.601	+0.227	—	—
features	motion_blur	high	15.6	1.000	0.197	0.476	—	—	-0.803	+0.280	—	—
features	salt_pepper	low	18.7	1.000	0.554	0.705	—	—	-0.446	+0.151	—	—
features	salt_pepper	med	11.8	1.000	0.404	0.692	—	—	-0.596	+0.288	—	—
features	salt_pepper	high	8.2	1.000	0.326	0.662	—	—	-0.674	+0.336	—	—
keypoints	gauss_noise	low	19.5	0.657	0.590	0.619	—	—	-0.068	+0.029	—	—
keypoints	gauss_noise	med	13.9	0.657	0.489	0.555	—	—	-0.168	+0.066	—	—
keypoints	gauss_noise	high	10.0	0.657	0.369	0.479	—	—	-0.288	+0.110	—	—
keypoints	motion_blur	low	20.8	0.657	0.578	0.620	—	—	-0.080	+0.043	—	—
keypoints	motion_blur	med	17.7	0.657	0.386	0.524	—	—	-0.272	+0.138	—	—
keypoints	motion_blur	high	15.6	0.657	0.159	0.387	—	—	-0.499	+0.229	—	—
keypoints	salt_pepper	low	18.7	0.657	0.590	0.623	—	—	-0.068	+0.034	—	—
keypoints	salt_pepper	med	11.8	0.657	0.438	0.614	—	—	-0.219	+0.176	—	—
keypoints	salt_pepper	high	8.2	0.657	0.348	0.598	—	—	-0.310	+0.250	—	—
segmentation	gauss_noise	low	19.5	0.410	0.359	0.386	—	—	-0.050	+0.027	—	—
segmentation	gauss_noise	med	13.9	0.410	0.294	0.350	—	—	-0.116	+0.056	—	—
segmentation	gauss_noise	high	10.0	0.410	0.209	0.299	—	—	-0.201	+0.090	—	—
segmentation	motion_blur	low	20.8	0.410	0.354	0.385	—	—	-0.056	+0.031	—	—
segmentation	motion_blur	med	17.7	0.410	0.231	0.311	—	—	-0.179	+0.081	—	—
segmentation	motion_blur	high	15.6	0.410	0.123	0.204	—	—	-0.287	+0.081	—	—
segmentation	salt_pepper	low	18.7	0.410	0.290	0.378	—	—	-0.120	+0.088	—	—
segmentation	salt_pepper	med	11.8	0.410	0.150	0.373	—	—	-0.260	+0.223	—	—
segmentation	salt_pepper	high	8.2	0.410	0.105	0.348	—	—	-0.304	+0.243	—	—

Bold = recovery ≥ +0.04 · “—” = not applicable (fine-tuning is detection-only) · same 1,521 images and GT in every row.

Additional evidence — remaining curves and per-class comparison



- Top: OKS AP and PQ vs SNR — same monotonic pattern as detection and features.
- Left: per-class detection AP under high Gaussian noise — 15 highest-clean-AP classes, same legend as slide 9.